SKRIPSI

PERANGKAT LUNAK PEMBUAT VISUALISASI GRAF SITUS WEB



Bram Matthew

NPM: 6182101043

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI DAN SAINS
UNIVERSITAS KATOLIK PARAHYANGAN
2026

DAFTAR ISI

Daftar Isi iii

Daftar Gambar v

1 Pendahuluan 1
- 1.1 Latar Belakang . . . 1
- 1.2 Rumusan Masalah . . . 1
- 1.3 Tujuan . . . 2
- 1.4 Batasan Masalah . . . 2
- 1.5 Metodologi . . . 2
- 1.6 Sistematika Pembahasan . . . 2

2 Landasan Teori 3
- 2.1 Hypertext Markup Language (HTML) . . . 3
  - 2.1.1 Pengertian HTML . . . 3
  - 2.1.2 Elemen dan Tag HTML . . . 3
  - 2.1.3 Struktur dasar HTML . . . 4
  - 2.1.4 Hierarki elemen HTML . . . 5
- 2.2 Document Object Model (DOM) . . . 6
  - 2.2.1 Pengertian DOM . . . 6
  - 2.2.2 Struktur DOM sebagai Tree . . . 6
  - 2.2.3 Jenis *node* pada DOM . . . 7
- 2.3 Jsoup . . . 8
  - 2.3.1 Pengertian Jsoup . . . 8
  - 2.3.2 Penggunaan Jsoup dalam Pemrosesan DOM . . . 8
- 2.4 Struktur Data Tree . . . 8
  - 2.4.1 Pengertian Tree . . . 8
  - 2.4.2 Tree pada Struktur DOM . . . 9
- 2.5 Struktur Data Graf . . . 9
  - 2.5.1 Pengertian Graf . . . 9
  - 2.5.2 Representasi Graf . . . 10
  - 2.5.3 Graf Berarah . . . 10
  - 2.5.4 Representasi DOM sebagai graf . . . 11
  - 2.5.5 Pengertian Traversal Graf . . . 11
  - 2.5.6 Depth-First Search . . . 12
- 2.6 Graphviz . . . 13
  - 2.6.1 Pengertian Graphviz . . . 13
  - 2.6.2 Format DOT . . . 13

Daftar Referensi 15

A Kode Program 17

DAFTAR GAMBAR

2.1 Gambar Tag pembuka dan penutup 4
2.2 Contoh Dom Tree 7
2.3 Contoh graf berarah sederhana (*node* dan *edge*) 11
2.4 Ilustrasi proses DFS pada Graf berarah 13
2.5 Visualisasi dari format DOT dengan menggunakan Graphviz 14

B.1 Hasil 1 19
B.2 Hasil 2 19
B.3 Hasil 3 19
B.4 Hasil 4 19

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi web yang sangat pesat menyebabkan halaman web modern memiliki struktur yang semakin kompleks. Setiap halaman web dibangun menggunakan *Hypertext Markup Language* (HTML) yang secara umum dapat direpresentasikan dalam bentuk struktur hierarkis yang dikenal sebagai *Document Object Model* (DOM). Struktur DOM ini berbentuk pohon (*tree*) yang terdiri dari elemen-elemen HTML yang saling terhubung melalui hubungan induk (*parent*) dan anak (*child*)[1].

Meskipun struktur DOM merupakan konsep dasar dalam pengembangan web, representasinya dalam bentuk kode HTML sering kali sulit dipahami secara langsung, terutama pada halaman web yang memiliki banyak elemen. Oleh karena itu, diperlukan suatu cara yang dapat membantu menyajikan struktur tersebut dalam bentuk yang lebih mudah dipahami.

Salah satu cara yang dapat digunakan adalah dengan memvisualisasikan struktur halaman web dalam bentuk graf. Dalam pendekatan ini, setiap elemen HTML direpresentasikan sebagai simpul (*node*) dan hubungan antar elemen direpresentasikan sebagai sisi (*edge*). Dengan demikian, struktur DOM dapat dilihat secara lebih jelas dan terstruktur.

Berdasarkan permasalahan tersebut, tugas akhir ini berfokus pada pengembangan perangkat lunak yang mampu mengambil dokumen HTML dari suatu *Uniform Resource Locator* (URL), membangun struktur DOM, dan merepresentasikannya dalam bentuk graf yang dapat divisualisasikan. Perangkat lunak ini diharapkan dapat membantu pengguna dalam memahami struktur halaman web secara lebih mudah.

1.2 Rumusan Masalah

Rumusan Masalah yang akan dibahas pada tugas akhir ini adalah:

1. Bagaimana cara mengambil dokumen HTML dari suatu situs web berdasarkan URL serta membangun struktur DOM?
2. Bagaimana cara merepresentasikan struktur DOM ke dalam bentuk graf yang terdiri dari (*node*) dan (*edge*) melalui proses traversal yang sistematis?
3. Bagaimana merancang perangkat lunak yang mampu menghasilkan visualisasi graf dari struktur halaman web secara otomatis dan mudah dipahami pengguna?

1.3 Tujuan

Tujuan yang ingin dicapai skripsi ini adalah sebagai berikut:

1. Mengambil dokumen HTML dari suatu situs web berdasarkan URL serta membangun struktur DOM.
2. Merepresentasikan struktur DOM ke dalam bentuk graf yang terdiri dari (*node*) dan (*edge*) melalui proses traversal yang sistematis.
3. Merancang perangkat lunak yang mampu menghasilkan visualisasi graf dari struktur halaman web secara otomatis dan mudah dipahami pengguna.

1.4 Batasan Masalah

Pada pengerjaan tugas akhir ini terhadap batasan sebagai berikut:

- Perangkat lunak dikembangkan menggunakan bahasa pemrograman Java

1.5 Metodologi

Metodologi pengerjaan tugas akhir ini adalah sebagai berikut:

1. Melakukan studi literatur mengenai HTML, DOM, struktur data graf, Graphviz, dan metode traversal.
2. Melakukan analisis kebutuhan serta perancangan arsitektur perangkat lunak yang akan dikembangkan.
3. Mengimplementasikan sistem untuk mengambil dokumen HTML, membangun struktur DOM, serta merepresentasikannya dalam bentuk graf.
4. Melakukan pengujian dan eksperimen menggunakan beberapa halaman web dengan struktur yang berbeda.
5. Melakukan analisis terhadap hasil visualisasi yang dihasilkan oleh sistem.
6. Menyusun dokumentasi dan laporan tugas akhir.

1.6 Sistematika Pembahasan

Sistematika pembahasan skripsi ini adalah sebagai berikut:

- Bab 1: Pendahuluan Membahas latar belakang, rumusan masalah, tujuan, batasan masalah, metodologi, dan sistematika pembahasan.
- Bab 2: Landasan Teori Membahas dasar dan hierarki HTML, struktur dan jenis node DOM, Graf berarah, DOT file dan Graphviz.
- Bab 3: Analisis
- Bab 4: Perancangan
- Bab 5: Implementasi dan Pengujian
- Bab 6: Kesimpulan dan Saran

BAB 2

LANDASAN TEORI

2.1 Hypertext Markup Language (HTML)

2.1.1 Pengertian HTML

Hypertext Markup Language (HTML) merupakan bahasa *markup* yang digunakan untuk menyusun struktur dasar halaman web. HTML tidak termasuk bahasa pemrograman karena tidak memiliki mekanisme logika seperti percabangan, perulangan, atau pengolahan data secara langsung, melainkan digunakan untuk mendeskripsikan struktur dan makna konten yang akan ditampilkan pada *browser* [2]. HTML memungkinkan pengembang mengatur berbagai jenis konten, seperti teks, gambar, tautan, tabel, formulir, serta elemen multimedia lainnya dalam sebuah dokumen web.

HTML berfungsi sebagai kerangka utama halaman web yang menentukan bagaimana setiap bagian konten disusun sehingga dapat ditampilkan secara terstruktur oleh *browser* [2]. Dalam pengembangan web, HTML umumnya digunakan bersama dengan CSS dan JavaScript. HTML berperan dalam membentuk struktur konten, CSS digunakan untuk mengatur tampilan atau gaya visual, sedangkan JavaScript digunakan untuk menambahkan interaktivitas pada halaman web.

HTML bekerja dengan konsep *tag-based structure*, yaitu setiap elemen ditandai dengan *tag* tertentu. Tag tersebut memberikan informasi kepada *browser* mengenai jenis dan fungsi konten, misalnya paragraf, judul, gambar, tautan, atau daftar. Melalui struktur ini, *browser* dapat memahami peran masing-masing elemen, menyusun hubungan antar elemen, dan menampilkannya dengan benar kepada pengguna.

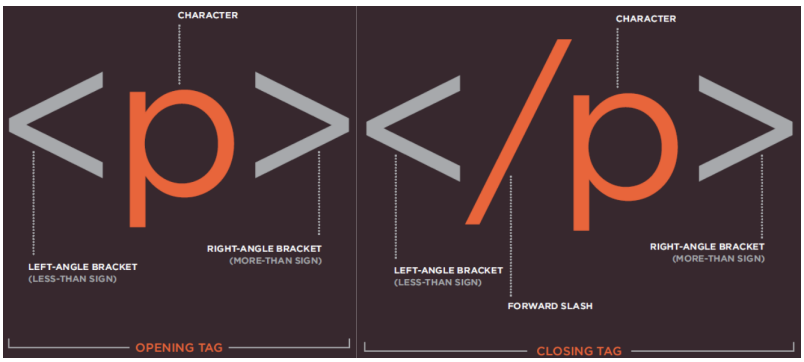
2.1.2 Elemen dan Tag HTML

Elemen HTML merupakan komponen dasar dalam penyusunan halaman web. Setiap elemen digunakan untuk merepresentasikan bagian tertentu dari konten, seperti paragraf, judul, gambar, tautan, tabel, daftar, maupun formulir. Secara umum, sebuah elemen HTML terdiri dari *tag* pembuka, isi elemen, dan *tag* penutup. Sebagai contoh, elemen paragraf dituliskan dengan sebagai *tag* pembuka, isi teks sebagai konten, dan sebagai *tag* penutup seperti pada gambar 2.1.

Tag berfungsi sebagai penanda yang memberi tahu *browser* mengenai jenis elemen yang sedang digunakan. Tag pembuka menunjukkan awal suatu elemen, sedangkan *tag* penutup menunjukkan akhir elemen tersebut. Namun, tidak semua elemen HTML memiliki *tag* penutup. Beberapa elemen, seperti <img>,
, dan <hr>, termasuk elemen kosong karena tidak memiliki isi di antara *tag* pembuka dan *tag* penutup.

Selain *tag*, elemen HTML juga dapat memiliki atribut yang memberikan informasi tambahan terhadap elemen tersebut. Atribut dituliskan di dalam *tag* pembuka dan umumnya terdiri dari nama atribut serta nilai atribut. Sebagai contoh, atribut *href* pada elemen <a> digunakan untuk menentukan alamat tautan, sedangkan atribut *src* pada elemen <img> digunakan untuk menentukan lokasi berkas gambar. Atribut lain seperti *id* dan *class* sering digunakan untuk memberi identitas atau pengelompokan elemen agar dapat diatur lebih lanjut menggunakan CSS atau diproses melalui JavaScript.

Struktur elemen yang saling bersarang (*nested elements*) memungkinkan terbentuknya hubungan hierarkis antar elemen. Dalam struktur ini, suatu elemen dapat menjadi induk bagi elemen lain yang berada di dalamnya. Misalnya, elemen <body> dapat memuat elemen , kemudian elemen tersebut dapat memuat elemen <p>, <img>, atau <a>. Hubungan bertingkat ini menjadi dasar penting dalam pembentukan struktur DOM karena *browser* akan membaca dokumen HTML sebagai susunan *node* yang saling terhubung.



Gambar 2.1: Contoh *tag* pembuka dan penutup dokumen HTML

2.1.3 Struktur dasar HTML

Dokumen HTML memiliki struktur standar yang terdiri dari beberapa bagian utama, yaitu <html>, <head>, dan <body>. Pada kode 2.2, struktur tersebut diawali dengan deklarasi <!DOCTYPE html> yang berfungsi untuk memberi tahu *browser* bahwa dokumen menggunakan standar HTML modern. Setelah deklarasi tersebut, elemen <html> digunakan sebagai *root* dari seluruh dokumen HTML dan menjadi wadah bagi elemen <head> serta <body>.

Bagian <head> pada kode berikut berisi informasi yang tidak ditampilkan secara langsung pada halaman, tetapi dibutuhkan oleh *browser* untuk memahami dokumen. Elemen <meta charset="UTF-8"> digunakan untuk menentukan pengkodean karakter, elemen <meta name="viewport"> digunakan untuk mengatur tampilan halaman pada perangkat yang berbeda, sedangkan elemen <title> digunakan untuk menentukan judul dokumen.

Bagian <body> pada kode berikut berisi konten utama yang akan ditampilkan kepada pengguna. Di dalamnya terdapat elemen <h1> sebagai judul utama, elemen <h2> sebagai subjudul, serta elemen <p> sebagai paragraf. Susunan tersebut menunjukkan bahwa isi halaman web ditempatkan di dalam <body> dan disusun berdasarkan fungsi masing-masing elemen.

Dengan demikian, kode berikut memperlihatkan bahwa dokumen HTML memiliki susunan yang hierarkis. Elemen <html> menjadi elemen induk, sedangkan <head> dan <body> menjadi elemen anak. Di dalam <body>, *heading* dan paragraf menjadi bagian dari konten halaman. Susunan

yang teratur ini memudahkan *browser* membaca dokumen, menampilkan halaman secara benar, dan membentuk DOM karena setiap elemen memiliki posisi serta hubungan yang jelas di dalam hierarki dokumen.

Kode 2.1: Contoh struktur dasar dokumen HTML yang terdiri dari deklarasi dokumen; elemen html; bagian head; yang berisi metadata dan title serta bagian body berisi heading dan paragraf sebagai konten utama halaman

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
</head>
    <body>
        <h1>This is the Main Heading</h1>
        <p>This text might be an introduction to the rest of the page, And if the page is a long one it might be split up into
            several sub-headings.</p>
            <h2>This is a Sub-Heading</h2>
            <p>Many long articles have sub-heading so to help you follow the structure of what is being written. There may
                even be sub-sub-headings (or lower-level headings).</p>
            <h2>Another Sub-Heading</h2>
            <p>Here you can see another sub-heading</p>
    </body>
</html>
```

2.1.4 Hierarki elemen HTML

Struktur HTML bersifat hierarkis, yang berarti elemen dapat berada di dalam elemen lain. Hubungan antar elemen ini membentuk struktur bertingkat yang menyerupai pohon, di mana setiap elemen memiliki posisi tertentu dalam dokumen. Dalam struktur tersebut, elemen yang memuat elemen lain disebut sebagai *parent*, sedangkan elemen yang berada di dalamnya disebut sebagai *child*. Elemen-elemen yang berada pada tingkat yang sama dan memiliki *parent* yang sama disebut sebagai *sibling*.

Sebagai contoh, elemen <body> dapat berperan sebagai *parent* bagi elemen . Selanjutnya, elemen dapat berisi elemen <p>, <img>, atau <a> sebagai *child*. Jika di dalam satu elemen terdapat beberapa elemen <p>, maka elemen-elemen tersebut dapat disebut sebagai *sibling* karena berada pada tingkat hierarki yang sama. Hubungan seperti ini menunjukkan bahwa dokumen HTML tidak hanya tersusun secara berurutan, tetapi juga memiliki hubungan induk-anak yang jelas antar elemen.

Hierarki ini penting karena menentukan bagaimana elemen saling berhubungan dan bagaimana struktur tersebut akan diproses menjadi DOM. Ketika *browser* membaca dokumen HTML, setiap elemen akan direpresentasikan sebagai *node* dalam struktur DOM. Hubungan *parent*, *child*, dan *sibling* pada HTML akan menjadi dasar pembentukan hubungan antar *node* di dalam DOM. Oleh karena itu, susunan elemen yang benar akan memudahkan *browser* dalam memahami struktur dokumen dan menampilkan halaman web secara sesuai.

Semakin kompleks halaman web, maka semakin dalam dan luas struktur hierarki yang terbentuk. Halaman yang memiliki banyak bagian, seperti header, navigasi, konten utama, sidebar, formulir, dan footer, akan menghasilkan struktur HTML yang lebih bercabang. Struktur hierarkis tersebut juga memengaruhi proses penelusuran elemen, baik dalam manipulasi DOM menggunakan JavaScript maupun dalam analisis struktur halaman menggunakan pendekatan *traversal* seperti *Depth First Search* (DFS).

2.2 Document Object Model (DOM)

2.2.1 Pengertian DOM

Document Object Model (DOM) adalah representasi dari dokumen HTML dalam bentuk struktur objek yang dapat diakses dan dimanipulasi oleh program. DOM memungkinkan setiap elemen dalam halaman web diperlakukan sebagai objek yang memiliki properti dan metode [1].

Menurut *World Wide Web Consortium*, DOM menyediakan antarmuka standar untuk mengakses dan memodifikasi dokumen HTML. Dengan adanya DOM, halaman web tidak hanya menjadi tampilan statis, tetapi juga dapat diproses dan dimodifikasi secara dinamis.

2.2.2 Struktur DOM sebagai Tree

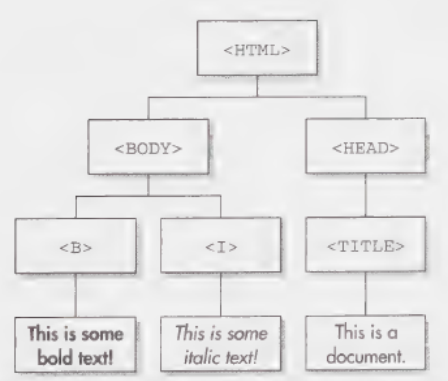
Struktur DOM berbentuk pohon (*tree structure*) karena dokumen HTML disusun dari elemen-elemen yang saling berhubungan secara hierarkis. Dalam DOM, setiap bagian dokumen direpresentasikan sebagai *node*, seperti *node* dokumen, *node* elemen, *node* atribut, dan *node* teks. Menurut Marini, DOM memodelkan dokumen sebagai kumpulan objek yang tersusun dalam struktur pohon sehingga setiap objek dapat diakses, ditelusuri, dan dimodifikasi melalui antarmuka pemrograman [1].

Pada struktur tersebut, *node* paling atas disebut *root node*. Untuk dokumen HTML, elemen <html> umumnya menjadi akar utama yang menaungi elemen <head> dan <body>. Elemen <head> dapat memiliki anak seperti <title> atau <meta>, sedangkan elemen <body> dapat memiliki anak seperti , <h1>, <p>, atau elemen lain yang membentuk isi halaman. Hubungan bertingkat ini membuat DOM dapat dipahami sebagai susunan induk dan anak antar elemen, sebagaimana ditunjukkan pada Gambar 2.2.

Setiap *node* dalam DOM dapat memiliki beberapa jenis relasi, yaitu:

- Parent (induk), yaitu *node* yang menaungi *node* lain di bawahnya.
- Child (anak), yaitu *node* yang berada langsung di dalam *node* induk.
- Sibling (saudara), yaitu *node* yang berada pada tingkat yang sama dan memiliki *parent* yang sama.

Relasi *parent*, *child*, dan *sibling* memungkinkan program menelusuri dokumen dari satu *node* ke *node* lain. Penelusuran dapat dilakukan dari *root* menuju *node* turunan, dari suatu *node* menuju *parent*, atau antar *node* yang berada pada level yang sama. Dengan bentuk pohon tersebut, struktur DOM dapat dianalisis secara sistematis menggunakan metode *traversal*, misalnya *Depth First Search* (DFS), untuk membaca seluruh elemen dari akar hingga *node-node* turunannya.



Gambar 2.2: Contoh Dom berbentuk *tree* yang berisi Body dan Head, di dalam Body terdapat elemen *bold* dan *italic*, di dalam Head terdapat elemen judul

2.2.3 Jenis *node* pada DOM

DOM tidak hanya terdiri dari elemen HTML, tetapi juga tersusun dari beberapa jenis *node* yang merepresentasikan bagian berbeda dari dokumen. DOM memandang dokumen sebagai kumpulan objek, sehingga setiap bagian dokumen dapat diperlakukan sebagai *node* yang memiliki jenis, nilai, serta hubungan tertentu dengan *node* lain [1]. Perbedaan jenis *node* ini penting karena program perlu mengetahui apakah suatu *node* merupakan elemen, teks, atribut, atau bagian dokumen lainnya sebelum dilakukan proses pembacaan maupun manipulasi.

Beberapa jenis *node* yang umum terdapat pada DOM antara lain:

- *Document node*, yaitu *node* utama yang merepresentasikan keseluruhan dokumen HTML. Node ini menjadi titik awal untuk mengakses struktur DOM.
- *Element node*, yaitu *node* yang merepresentasikan *tag* HTML seperti <html>, <head>, <body>, , <p>, atau <a>. *Node* jenis ini biasanya memiliki relasi *parent* dan *child* dengan elemen lain.
- *Text node*, yaitu *node* yang merepresentasikan isi teks yang berada di dalam suatu elemen. Misalnya, teks dalam elemen <p> akan dibaca sebagai *node* teks.
- *Attribute node*, yaitu *node* yang merepresentasikan atribut pada elemen HTML, seperti *id*, *class*, *href*, atau *src*. Atribut memberikan informasi tambahan terhadap elemen yang memilikinya.
- *Comment node*, yaitu *node* yang merepresentasikan komentar di dalam dokumen HTML. Node ini tidak ditampilkan sebagai konten utama halaman, tetapi tetap dapat menjadi bagian dari struktur dokumen.

Keberadaan berbagai jenis *node* tersebut membuat DOM lebih kaya dibandingkan HTML mentah karena DOM tidak hanya menyimpan *tag*, tetapi juga menyimpan teks, atribut, komentar, serta relasi antarbagian dokumen. Dengan demikian, analisis DOM dapat memberikan informasi yang lebih lengkap mengenai struktur halaman web. Pemahaman terhadap jenis *node* diperlukan agar elemen-elemen yang relevan dapat dipilih dan dipetakan ke dalam representasi graf secara tepat.

2.3 Jsoup

2.3.1 Pengertian Jsoup

Jsoup merupakan *library* Java yang digunakan untuk membaca, memproses, mengekstraksi, dan memanipulasi dokumen HTML. *Library* ini menyediakan antarmuka pemrograman yang memudahkan pengembang dalam mengambil data dari dokumen HTML, baik yang berasal dari berkas lokal, *string* HTML, maupun halaman web yang diakses melalui *URL* [3]. Dengan menggunakan Jsoup, struktur HTML dapat diproses menjadi bentuk dokumen yang lebih mudah ditelusuri dan dianalisis oleh program.

Jsoup bekerja dengan cara mem-parsing kode HTML dan membentuk representasi dokumen yang menyerupai struktur DOM. Setelah dokumen diproses, elemen-elemen HTML dapat diakses berdasarkan *tag*, atribut, teks, maupun relasi antar elemen. Hal ini membuat Jsoup banyak digunakan dalam pengolahan dokumen web, seperti ekstraksi informasi, pembersihan data HTML, analisis struktur halaman, dan pengambilan elemen tertentu dari sebuah halaman web.

Kode 2.2: Potongan kode dengan fungsi mengambil *homepage* Wikipedia lalu melakukan *parse* menjadi DOM dan memilih judul berita dari bagian "Berita" ke dalam daftar Elemen

```
    Document doc = Jsoup.connect("https://en.wikipedia.org/").get();
    log(doc.title());
    Elements newsHeadlines = doc.select("#mp-itn b a");
    for (Element headline : newsHeadlines) {
        log("%s\n\t%s",
            headline.attr("title"), headline.absUrl("href"));
}
```

2.3.2 Penggunaan Jsoup dalam Pemrosesan DOM

Dalam konteks pemrosesan DOM, Jsoup berperan sebagai alat bantu untuk mengubah dokumen HTML mentah menjadi objek dokumen yang dapat ditelusuri secara sistematis. Setiap elemen HTML yang telah diproses dapat diakses sebagai *node* yang memiliki hubungan dengan elemen lain, seperti *parent*, *child*, dan *sibling*. Relasi tersebut memungkinkan program membaca struktur halaman secara hierarkis tanpa harus memproses teks HTML secara manual.

Pada penelitian ini, Jsoup digunakan sebagai tahap awal untuk memperoleh struktur DOM dari dokumen HTML. Elemen-elemen yang dihasilkan oleh Jsoup dapat dipetakan menjadi *node*, sedangkan hubungan antar elemen seperti *parent* dan *child* dapat dipetakan menjadi *edge*. Dengan demikian, Jsoup menjadi penghubung antara dokumen HTML mentah dan representasi graf yang dapat dianalisis serta divisualisasikan.

2.4 Struktur Data Tree

2.4.1 Pengertian Tree

Tree adalah struktur data hierarkis yang terdiri dari sekumpulan *node* dan hubungan antar *node* [4]. Node paling atas dalam *tree* disebut *root*, sedangkan *node* yang berada di bawah *node* lain disebut *child*. Node yang memiliki *child* disebut *parent*, sementara *node* yang tidak memiliki *child* disebut

leaf. Struktur *tree* digunakan untuk merepresentasikan hubungan bertingkat, yaitu satu *node* dapat memiliki beberapa *node* turunan.

Secara konsep, *tree* merupakan bentuk khusus dari graf yang tidak memiliki siklus dan memiliki hubungan hierarkis yang jelas. Setiap *node* dalam *tree*, kecuali *root*, memiliki satu *parent*. Karakteristik ini membuat *tree* cocok digunakan untuk memodelkan struktur yang tersusun secara bertingkat, seperti struktur folder, organisasi data, ekspresi matematika, dan dokumen HTML.

2.4.2 Tree pada Struktur DOM

DOM memiliki karakteristik yang menyerupai *tree* karena setiap elemen dalam dokumen HTML tersusun secara hierarkis. Elemen <html> biasanya menjadi *root*, sedangkan elemen <head> dan <body> menjadi *child* dari elemen tersebut. Selanjutnya, elemen-elemen lain seperti , <p>, <img>, dan <a> dapat menjadi turunan dari elemen <body> atau elemen lain di dalamnya.

Pemahaman mengenai *tree* penting dalam penelitian ini karena struktur DOM yang berbentuk *tree* dapat ditelusuri menggunakan algoritma *traversal*. Salah satu algoritma yang sesuai untuk struktur seperti ini adalah DFS, karena DFS menelusuri *node* secara mendalam dari *root* menuju leaf sebelum kembali ke cabang lain. Dengan demikian, konsep *tree* menjadi dasar untuk memahami hubungan antara DOM, graf, dan proses *traversal*.

2.5 Struktur Data Graf

2.5.1 Pengertian Graf

Graf adalah struktur data yang terdiri dari simpul (*node* atau *vertex*) dan sisi (*edge*) yang menghubungkan simpul-simpul tersebut. Dalam kajian algoritma, graf umumnya dinyatakan sebagai pasangan himpunan $G = (V, E)$, dengan V sebagai himpunan simpul dan E sebagai himpunan sisi yang merepresentasikan hubungan antar simpul [4]. Setiap simpul dapat menggambarkan suatu objek, sedangkan sisi menunjukkan adanya relasi atau keterhubungan antara dua objek.

Menurut Cormen, graf banyak digunakan untuk memodelkan berbagai jenis hubungan dalam komputasi karena bentuknya fleksibel dan dapat merepresentasikan struktur yang sederhana maupun kompleks [4]. Sebagai contoh, simpul dapat merepresentasikan kota, halaman web, pengguna, proses, atau elemen dalam dokumen, sedangkan sisi dapat merepresentasikan jalan, tautan, hubungan sosial, komunikasi antar proses, atau hubungan induk-anak antar elemen.

Graf dapat dibedakan berdasarkan karakteristik sisinya. Pada graf tidak berarah, hubungan antar simpul tidak memiliki arah tertentu sehingga relasi antara dua simpul dianggap berlaku dua arah. Sebaliknya, pada graf berarah, setiap sisi memiliki arah dari satu simpul ke simpul lainnya. Selain itu, graf juga dapat memiliki bobot pada sisi untuk menyatakan nilai tertentu, seperti jarak, biaya, waktu, atau tingkat hubungan antar simpul.

Dalam konteks penelitian ini, konsep graf digunakan untuk merepresentasikan struktur DOM dari dokumen HTML. Setiap elemen HTML dapat dipandang sebagai simpul, sedangkan hubungan antara elemen induk dan elemen anak dapat dipandang sebagai sisi. Dengan pemodelan tersebut, struktur halaman web dapat dianalisis menggunakan konsep teori graf, seperti hubungan antar *node*, arah hubungan, dan proses *traversal*.

2.5.2 Representasi Graf

Representasi graf adalah cara menyimpan struktur graf agar dapat diproses oleh komputer. Dalam implementasi algoritma, graf tidak cukup hanya didefinisikan sebagai himpunan simpul dan sisi, tetapi perlu direpresentasikan dalam bentuk data tertentu. Dua bentuk representasi graf yang umum digunakan adalah adjacency matrix dan adjacency list [4].

Adjacency matrix merepresentasikan graf dalam bentuk matriks dua dimensi. Setiap baris dan kolom mewakili simpul, sedangkan nilai pada sel matriks menunjukkan ada atau tidaknya sisi antara dua simpul. Representasi ini mudah dipahami dan efisien untuk memeriksa hubungan langsung antar simpul, tetapi dapat menggunakan ruang penyimpanan yang besar jika jumlah simpul banyak dan jumlah sisi sedikit.

Adjacency list merepresentasikan graf dengan menyimpan daftar tetangga untuk setiap simpul. Setiap simpul memiliki daftar *node* lain yang terhubung dengannya. Representasi ini lebih hemat ruang untuk graf yang memiliki jumlah sisi relatif sedikit dibandingkan jumlah kemungkinan sisi. Dalam konteks DOM, adjacency list sesuai digunakan karena setiap elemen biasanya hanya memiliki hubungan langsung dengan sejumlah *child* tertentu, bukan dengan seluruh elemen dalam dokumen.

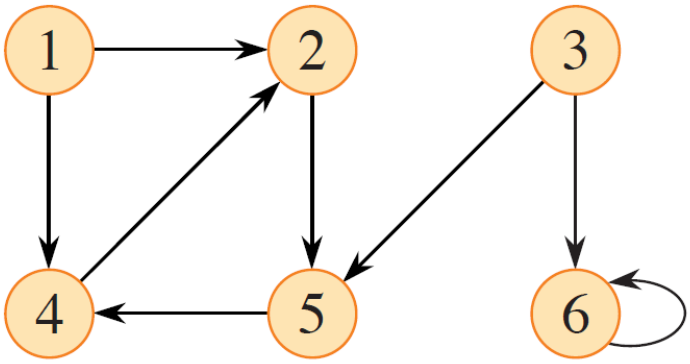
2.5.3 Graf Berarah

Graf berarah (*directed graph* atau *digraph*) adalah graf yang setiap sisinya memiliki arah tertentu. Pada graf berarah, sisi tidak hanya menunjukkan bahwa dua simpul saling terhubung, tetapi juga menunjukkan arah hubungan dari satu simpul menuju simpul lainnya. Secara formal, sisi pada graf berarah dapat dinyatakan sebagai pasangan terurut (u, v), yang berarti sisi tersebut berawal dari simpul u dan menuju simpul v [4].

Berbeda dengan graf tidak berarah, hubungan pada graf berarah tidak selalu berlaku dua arah. Jika terdapat sisi dari simpul u ke simpul v, maka hal tersebut tidak secara otomatis berarti terdapat sisi dari simpul v ke simpul u. Oleh karena itu, arah sisi menjadi informasi penting dalam memahami pola hubungan antar simpul. Konsep ini banyak digunakan untuk memodelkan hubungan yang memiliki alur, urutan, atau ketergantungan, seperti tautan antar halaman web, aliran data, proses kerja, maupun struktur hierarkis.

Dalam konteks DOM, hubungan antara *parent* dan *child* memiliki arah yang jelas, yaitu dari *node* induk menuju *node* anak. Misalnya, elemen <html> memiliki hubungan ke elemen <head> dan <body>, kemudian elemen <body> dapat memiliki hubungan ke elemen , <p>, atau elemen lainnya. Arah hubungan tersebut menunjukkan bahwa suatu elemen berada di bawah elemen lain dalam struktur dokumen.

Pemodelan DOM sebagai graf berarah membantu menunjukkan alur hubungan antar elemen secara lebih terstruktur. Dengan menggunakan arah sisi, proses penelusuran dapat dilakukan dari *root node* menuju *node-node* turunannya. Hal ini penting dalam analisis struktur halaman web karena urutan dan arah hubungan antar elemen memengaruhi cara DOM dibaca, divisualisasikan, dan ditelusuri menggunakan algoritma *traversal* seperti *Depth First Search*. Contoh bentuk graf berarah sederhana yang terdiri dari *node* dan *edge* ditunjukkan pada Gambar 2.3.



Gambar 2.3: Contoh graf berarah sederhana (*node* dan *edge*)

2.5.4 Representasi DOM sebagai graf

Struktur DOM dapat direpresentasikan sebagai graf dengan cara memetakan setiap elemen HTML sebagai *node* dan hubungan *parent-child* sebagai *edge*. Dalam representasi ini, elemen *root* seperti <html> menjadi *node* awal, sedangkan elemen-elemen turunannya seperti <head>, <body>, , <p>, dan elemen lain menjadi *node* yang terhubung melalui *edge*.

Karena hubungan antar elemen dalam DOM memiliki arah dari *parent* menuju *child*, struktur DOM lebih sesuai dimodelkan sebagai graf berarah. Setiap *edge* menunjukkan hubungan hierarkis antara dua elemen. Misalnya, hubungan dari <body> ke menunjukkan bahwa elemen berada di dalam elemen <body>. Dengan pemetaan ini, struktur HTML yang semula berbentuk dokumen teks dapat dianalisis sebagai struktur *node* dan *edge*.

Representasi DOM sebagai graf mempermudah proses analisis dan visualisasi struktur halaman web. Melalui pendekatan graf, hubungan antar elemen dapat ditelusuri menggunakan algoritma *traversal* seperti DFS. Selain itu, representasi ini juga mempermudah pembuatan visualisasi menggunakan Graphviz karena struktur DOM dapat diubah ke dalam format DOT dan ditampilkan sebagai diagram graf.

2.5.5 Pengertian Traversal Graf

Traversal graf adalah proses mengunjungi setiap *node* dalam graf secara sistematis berdasarkan aturan tertentu. Proses ini dilakukan untuk menelusuri hubungan antar *node* sehingga seluruh struktur graf dapat diproses tanpa ada *node* yang terlewat. Dalam kajian algoritma, *traversal* menjadi langkah penting karena banyak permasalahan graf membutuhkan proses pemeriksaan *node* dan *edge*, misalnya untuk mencari jalur, menentukan keterhubungan, menemukan komponen, atau menganalisis struktur hubungan antar objek [4].

Pada proses *traversal*, suatu *node* biasanya dipilih sebagai titik awal, kemudian algoritma akan menelusuri *node-node* lain yang terhubung melalui *edge*. Urutan kunjungan *node* bergantung pada metode *traversal* yang digunakan. Beberapa metode *traversal* yang umum digunakan adalah *Breadth-First Search* (BFS) dan *Depth-First Search* (DFS). BFS menelusuri *node* berdasarkan tingkat kedekatan dari *node* awal, sedangkan DFS menelusuri *node* secara mendalam hingga mencapai cabang terdalam sebelum kembali ke *node* sebelumnya.

Dalam konteks penelitian ini, *traversal* graf digunakan untuk menelusuri struktur DOM yang telah direpresentasikan sebagai graf. Setiap elemen HTML dipandang sebagai *node*, sedangkan hubungan *parent-child* dipandang sebagai *edge*. Dengan *traversal*, struktur halaman web dapat dibaca secara teratur dari *node* awal menuju *node-node* turunannya. Hal ini membantu proses analisis DOM karena setiap elemen dapat diperiksa sesuai urutan hubungan hierarkisnya.

2.5.6 Depth-First Search

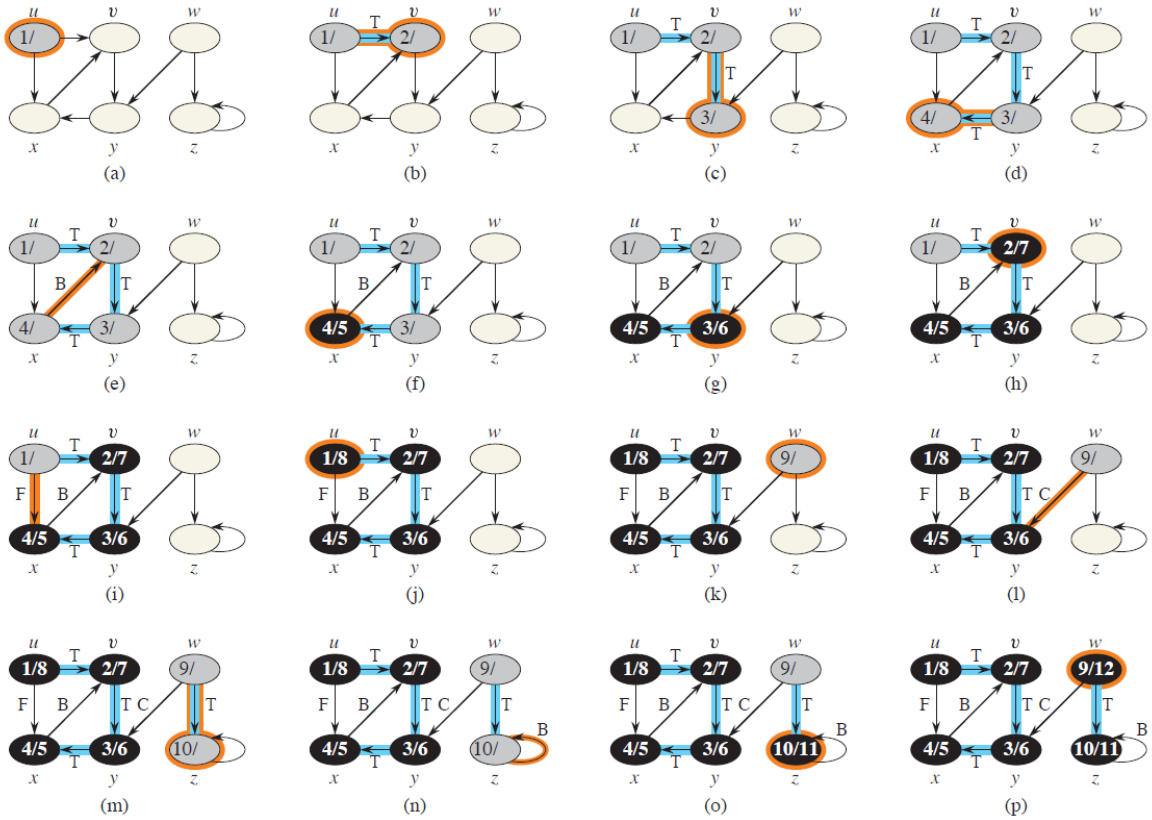
Depth-First Search (DFS) adalah metode *traversal* graf yang bekerja dengan menelusuri suatu simpul hingga ke bagian terdalam terlebih dahulu sebelum kembali ke simpul sebelumnya. Sesuai dengan namanya, DFS akan memilih untuk bergerak lebih dalam pada graf selama masih terdapat sisi yang belum dieksplorasi dari simpul yang sedang dikunjungi.

Menurut Cormen, DFS mengeksplorasi sisi-sisi yang keluar dari simpul yang paling baru ditemukan dan masih memiliki sisi yang belum diperiksa. Jika seluruh sisi dari simpul tersebut telah dieksplorasi, proses pencarian akan melakukan *backtracking*, yaitu kembali ke simpul asal tempat simpul tersebut ditemukan untuk melanjutkan eksplorasi sisi lain yang belum dikunjungi [4]. Proses ini berlanjut hingga seluruh simpul yang dapat dicapai dari simpul awal berhasil ditemukan. Apabila masih terdapat simpul lain yang belum ditemukan, DFS akan memilih simpul tersebut sebagai sumber baru dan mengulangi proses pencarian sampai seluruh simpul dalam graf telah diperiksa.

Dalam prosesnya, DFS juga mencatat hubungan pendahulu (*predecessor*) dari setiap simpul yang ditemukan. Ketika suatu simpul baru ditemukan dari simpul lain, hubungan tersebut dicatat sebagai bagian dari struktur hasil pencarian. Pada graf yang tidak seluruh simpulnya terhubung dari satu sumber, DFS dapat menghasilkan beberapa pohon pencarian, sehingga hasil keseluruhannya disebut sebagai *depth-first forest*. Sisi-sisi yang membentuk hubungan pendahulu tersebut dikenal sebagai *tree edges* [4].

DFS menggunakan penandaan warna untuk menunjukkan status setiap simpul selama proses *traversal*. Simpul yang belum dikunjungi diberi warna putih, simpul yang sedang diproses diberi warna abu-abu, dan simpul yang seluruh sisi keluarnya telah selesai diperiksa diberi warna hitam. Selain itu, DFS juga mencatat dua waktu pada setiap simpul, yaitu waktu penemuan (*discovery time*) ketika simpul pertama kali dikunjungi dan waktu penyelesaian (*finishing time*) ketika seluruh daftar ketetanggaan simpul tersebut telah selesai diperiksa. Informasi ini berguna untuk memahami urutan *traversal* dan struktur hubungan antar simpul dalam graf [4].

Metode ini sangat cocok untuk struktur berbentuk *tree* seperti DOM karena mengikuti pola *parent-child* secara mendalam. Pada konteks DOM, DFS dapat digunakan untuk menelusuri elemen dari *node* induk ke *node* anak secara bertahap hingga mencapai elemen terdalam, kemudian kembali ke *node* sebelumnya untuk melanjutkan penelusuran ke cabang lain. Ilustrasi proses DFS pada graf berarah dapat dilihat pada Gambar 2.4. Oleh karena itu, DFS sering digunakan untuk eksplorasi struktur hierarkis dan graf [4].



Gambar 2.4: Ilustrasi proses DFS pada graf berarah. Sisi-sisi diklasifikasikan sesuai urutan penjelajahannya: sisi pohon diberi *label* T, sisi belakang B, sisi depan F, dan sisi silang C.

2.6 Graphviz

2.6.1 Pengertian Graphviz

Graphviz adalah perangkat lunak visualisasi grafik *open source*. Visualisasi grafik adalah cara untuk merepresentasikan informasi struktural sebagai diagram grafik abstrak dan jaringan. Teknologi ini memiliki aplikasi penting dalam jaringan, bioinformatika, rekayasa perangkat lunak, desain basis data dan web, pembelajaran mesin, dan antarmuka visual untuk domain teknis lainnya[5].

Menurut *Graphviz Project*, Graphviz menyediakan berbagai algoritma *layout* untuk menampilkan graf secara otomatis.

2.6.2 Format DOT

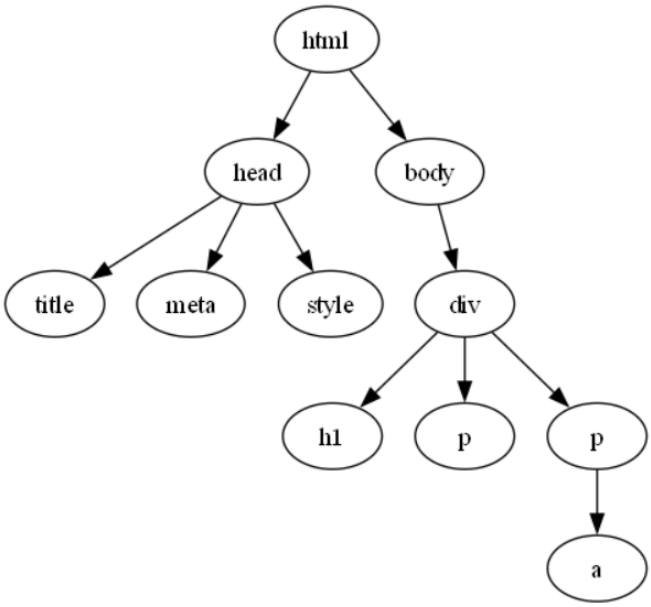
Graphviz menggunakan format DOT untuk mendefinisikan graf. DOT merupakan bahasa deskripsi graf berbasis teks yang digunakan untuk menuliskan struktur *node* dan *edge* secara sederhana. Melalui format ini, pengguna dapat menentukan jenis graf, nama graf, hubungan antar *node*, serta atribut visual seperti *label*, warna, bentuk *node*, dan arah hubungan. Graf tidak berarah pada DOT didefinisikan menggunakan kata kunci *graph* dengan simbol –, sedangkan graf berarah didefinisikan menggunakan kata kunci *digraph* dengan simbol ->.

Format DOT sesuai digunakan untuk merepresentasikan struktur DOM karena hubungan antar elemen HTML bersifat hierarkis dan berarah dari *parent* menuju *child*. Misalnya, hubungan antara

elemen HTML dan *head* dapat dituliskan sebagai HTML -> *head*, yang menunjukkan adanya *edge* dari *node* HTML menuju *node head*. Contoh penulisan struktur DOM dalam format DOT ditunjukkan pada Kode 2.3, sedangkan hasil visualisasinya menggunakan Graphviz ditunjukkan pada Gambar 2.5. Dalam konteks penelitian ini, DOT digunakan sebagai format perantara untuk mengubah struktur DOM menjadi representasi graf yang kemudian dapat divisualisasikan menggunakan Graphviz.

Kode 2.3: Contoh kode .dot

```
digraph DOM {
node [shape=oval];
node1 [label="html"];
node2 [label="head"];
node1 -> node2;
node3 [label="title"];
node2 -> node3;
node4 [label="meta"];
node2 -> node4;
node5 [label="style"];
node2 -> node5;
node6 [label="body"];
node1 -> node6;
node7 [label="div"];
node6 -> node7;
node8 [label="h1"];
node7 -> node8;
node9 [label="p"];
node7 -> node9;
node10 [label="p"];
node7 -> node10;
node11 [label="a"];
node10 -> node11;
}
```



Gambar 2.5: Visualisasi dari kode 2.3 dengan format DOT dengan menggunakan Graphviz

DAFTAR REFERENSI

[1] Marini, J. (2002) *The Document Object Model*. McGraw-Hill/Osborne Media, America.

[2] Duckket, J. (2011) *HTML and CSS, Design and Build Websites*. John Wiley and Sons, Inc, Canada.

[3] Hedley, J. dan jsoup contributors (2026) jsoup: Java html parser.

[4] Cormen, T. H., Leiserson, C. E., Rivest, R. L., dan Stein, C. (2022) *Introduction to Algorithms*, 4th edition. The MIT Press, America.

[5] Ellson, J., Gansner, E., Hu, Y., North, S., Jacobsson, M., Fernandez, M., dan Hansen, M. (2024) Graphviz. https://graphviz.org/. 28 April 2026.

LAMPIRAN A

KODE PROGRAM

Kode A.1: MyCode.c

```c

// This does not make algorithmic sense,
// but it shows off significant programming characters.

#include<stdio.h>

void myFunction( int input, float* output ) {
    switch ( array[i] ) {
        case 1: // This is silly code
            if ( a >= 0 || b <= 3 && c != x )
                *output += 0.005 + 20050;
            char = 'g';
            b = 2^n + ~right_size - leftSize * MAX_SIZE;
            c = (--aaa + &daa) / (bbb++ - ccc % 2 );
            strcpy(a,"hello $@?");
    }
    count = ~mask | 0x00FF00AA;
}

// Fonts for Displaying Program Code in LATEX
// Adrian P. Robson, nepsweb.co.uk
// 8 October 2012
// http://nepsweb.co.uk/docs/progfonts.pdf
```

Kode A.2: MyCode.java

```java
import java.util.ArrayList;
import java.util.Collections;
import java.util.HashSet;

//class for set of vertices close to furthest edge
public class MyFurSet {
    protected int id;                                   //id of the set
    protected MyEdge FurthestEdge;                      //the furthest edge
    protected HashSet<MyVertex> set;                    //set of vertices close to furthest edge
    protected ArrayList<ArrayList<Integer>> ordered;    //list of all vertices in the set for each trajectory
    protected ArrayList<Integer> closeID;               //store the ID of all vertices
    protected ArrayList<Double> closeDist;              //store the distance of all vertices
    protected int totaltrj;                             //total trajectories in the set

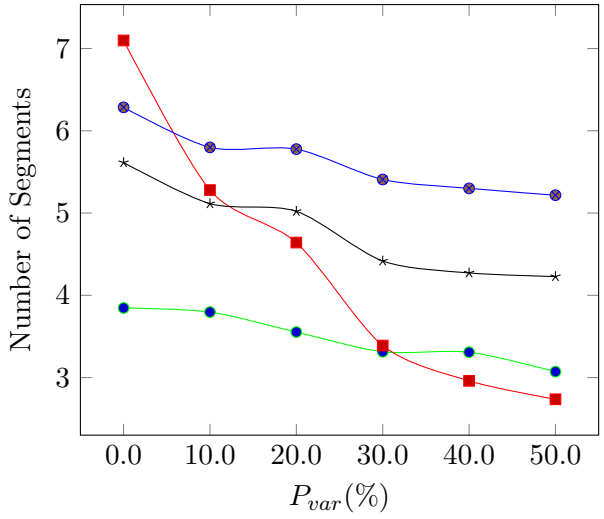
    /*
     * Constructor
     * @param id : id of the set
     * @param totaltrj : total number of trajectories in the set
     * @param FurthestEdge : the furthest edge
     */
    public MyFurSet(int id,int totaltrj,MyEdge FurthestEdge) {
        this.id = id;
        this.totaltrj = totaltrj;
        this.FurthestEdge = FurthestEdge;
        set = new HashSet<MyVertex>();
        ordered = new ArrayList<ArrayList<Integer>>();
        for (int i=0;i<totaltrj;i++) ordered.add(new ArrayList<Integer>());
        closeID = new ArrayList<Integer>(totaltrj);
        closeDist = new ArrayList<Double>(totaltrj);
        for (int i = 0;i <totaltrj;i++) {
            closeID.add(-1);
            closeDist.add(Double.MAX_VALUE);
        }
    }

}
```

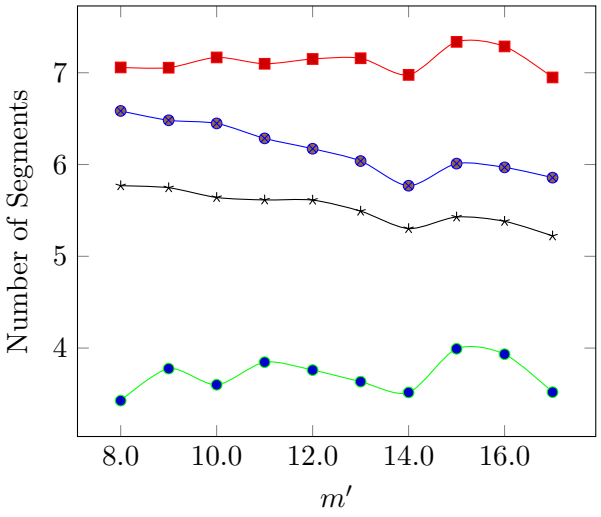
LAMPIRAN B

HASIL EKSPERIMEN

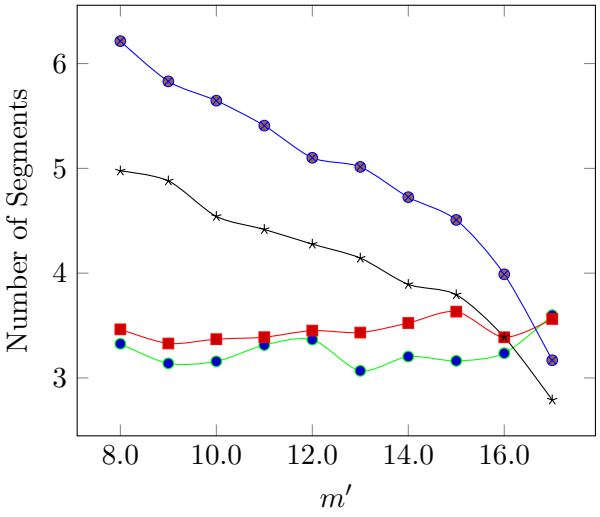
Hasil eksperimen berikut dibuat dengan menggunakan TIKZPICTURE (bukan hasil excel yg diubah ke file bitmap). Sangat berguna jika ingin menampilkan tabel (yang kuantitasnya sangat banyak) yang datanya dihasilkan dari program komputer.



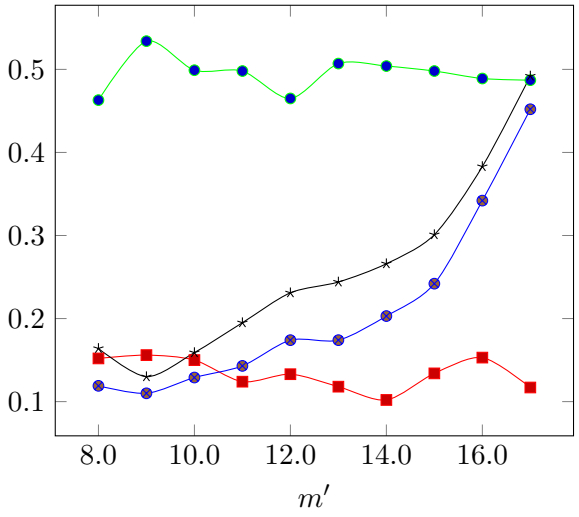
Gambar B.1: Hasil 1



Gambar B.2: Hasil 2



Gambar B.3: Hasil 3



Gambar B.4: Hasil 4